Security Now! #1085 - 06-30-26

A SOTA State-Sponsored Campaign

This week on Security Now!

• Win10's popularity forces another year of free updates. • CISA directs all federal agencies to update their UniFi OS devices. • CISA gave federal agencies "the weekend" to update Cisco devices. • Australia is disturbed by a deeply compromised infrastructure provider. • OpenAI introduces Daybreak-powered "Patch the Planet" initiative. • Meta's employee monitoring-for-AI-training backfired badly. • Script Kiddies figure out how to use AI to find vulnerabilities. • AI improves with "looping", "repeating" or "iterating". • A wonderful story about Kevin Mitnick. • Serious hackers mistakenly left a server directory accessible.

How to create a dead-end for cyclists:

Security News

Windows 10 ESU updates extended another year

It's gratifying to see a prediction about something that really should be done, come true. Sadly, gratifying or not, that doesn't happen often enough. Our listeners all know how disgusted I've been with Microsoft's continuing attempts to squeeze their Windows 10 users into moving to Windows 11. Many – and for quite some time, most – current Win10 users evidenced just as little desire to do that as once upon a time Windows 7 users wanted to move to Windows 8. Thanks, but no thanks. Everything is working fine, go away.

As we also know, Microsoft has arbitrarily, capriciously and unnecessarily raised the minimum hardware requirements for Windows 11 in a transparent effort to force the purchase of new and now onerously expensive PCs. We know this was arbitrary, capricious and unnecessary because Windows 11 runs quite well without complaint on PC hardware that lacks every one of those newly imposed so-called "requirements." They can all be bypassed because none of them are actually required.

Against this backdrop, in the summer of 2025 – June 24th to be exact – Microsoft reminded everyone that all support for Windows 10 would be ending a few months from then, on October 14th, 2025. There was only one problem with that: Still, no one wanted Windows 11 and nearly everyone was still quite happily using Windows 10. So, as we covered at the time, Microsoft blinked and gave everyone an additional full year of ESU Extended Service Updates <quote> *"in order to give everyone more time to migrate to Windows 11."* – or apparently, quite often, to give everyone more time to save up the money needed to purchase a new PC when the one they currently had was running Windows 10 just fine. So this allowed everyone to remain on the ESU plan until October 12th of 2026, later this year.

We're back to beating this poor and quite dead horse because time flies and we're once again at the end of June and, still, despite reluctantly returning feature after feature to Windows 11 that had been removed from Windows 10, and even after reverting some of the incredibly inefficient user-interface implementations that had been largely responsible for Windows 11 poor performance, no one still wants Windows 11 and it still won't voluntarily run on the hardware that's currently running Windows 10 just fine. And on top of all that, thanks to the AI drama that has swept the globe, that new Windows 11-capable PC will be significantly more expensive to purchase today than it would have been a year ago.

So can you guess what Microsoft just did? Yep. **Blink!!** They once again extended the Windows 10 ESU program for another year, until October 12th of 2027. Everyone using Windows 10 gets to keep using the Windows they love on the machines they already have. And what's even better is that the continuation of the ESU program means that Windows 10 can be the recipient of the results of Microsoft's still unnamed "Codename MDASH" system, which will be cleaning up the mess that was left behind by decades of Microsoft's previous human developers.

So, it's really the best of all possible worlds. Since all development on Windows 10 was blessedly halted years ago, Microsoft will no longer be introducing more new bugs than they remove every month. Instead, the extension of the ESU program for another year will give their new AI- model driven bug discovery and removal system the time it requires to remove the thousands of latent bugs Windows still carries, thus turning Windows 10 into a near-perfect operating system.

Thank you, Microsoft.

Given where we are today I'll make another prediction: Given that the current RAM and semiconductor shortage is expected to endure into 2028, this is likely to hold PC prices high. Since the recent performance improvements in Windows 11 are finally beginning to allow it to run as well as Windows 10 always has on the same hardware, and since it has always been able to run on that same hardware, I expect we're going to see some form of "Junior 11" which will, for face-saving reasons, strip out some features – like Recall and some of the Copilot+ AI crap – and will therefore "surprise!" be able to run anywhere Windows 10 can. Ultimately, this is going to be the only way for Microsoft to move the remainder of their holdout users over to Windows 11. It'll cost no one anything and it will get everyone back under the same codebase, which is where Microsoft really does need them to be over the long run.

CISA gives Federal Ubiquity users 3 days under the BOD

A quick follow-up on the state of the recent Ubiquity flaws. Since CISA has seen hackers actively exploiting the three flaws in Ubiquity's UniFi OS, last Wednesday, CISA gave all federal agencies 3 days to apply the available security updates or recommended mitigations.

Three Ubiquiti flaws have been added to CISA's KEV – Known Exploited Vulnerabilities database:

- CVE-2026-34908: is an access control bypass flaw that allows an unauthenticated attacker to make unauthorized changes to a UniFi OS system, potentially leading to full system compromise.
- CVE-2026-34909: a directory/path traversal vulnerability that allows an attacker to access sensitive files on the underlying operating system, potentially exposing configuration files, credentials, and other sensitive data that could facilitate account takeover.
- CVE-2026-34910: an improper input validation flaw that enables an attacker to inject and execute arbitrary operating system commands, potentially leading to remote code execution and complete system takeover.

As we know, Ubiquiti released updates for all three vulnerabilities in May, and Leo's UniFi OS instances, which were all set to auto update, all did. So he was never in any danger. Hopefully, everyone else has done this too. What's changed is that the pace of attacks following disclosure and/or reverse engineering of update releases necessitates taking the human out of the update decision loop. Just let automation handle that. Might it screw up? That's a possibility. But the incidences of such screw-ups have always been rare and we can expect them to become more rare as more of our infrastructure is secured.

CISA gives Federal Cisco users "the weekend" to update

Last Friday we learned that those 3-day CISA BOD – Binding Operational Directive – "thou must update!" notices include the weekend. CISA issued a directive Friday, giving federal agencies until Sunday night to patch. (Come to think of it, it might be that patching over a weekend is easier, since the network would probably be much quieter with fewer, if any, people disturbed by an updated and rebooted system.)

The story behind this one is a high-severity SSRF (server-side request forgery) vulnerability, tracked as CVE-2026-20230, which was discovered in Cisco's Unified Communications Manager Server. Cisco released security updates to address the 20230 flaw three weeks earlier, on June 3rd and at the time warned that exploitation could give attackers root privileges on the device. They wrote: "A vulnerability in Cisco Unified Communications Manager (Unified CM) and Cisco Unified Communications Manager Session Management Edition (Unified CM SME) could allow an unauthenticated, remote attacker to conduct server-side request forgery (SSRF) attacks through an affected device. This vulnerability is due to improper input validation for specific HTTP requests. An attacker could exploit this vulnerability by sending a crafted HTTP request to an affected device. A successful exploit could allow the attacker to write files to the underlying operating system that could be used later to elevate to root." So that was then. Three weeks later, the vulnerability is now being actively exploited.

The good news is, exploitation took nearly three weeks. So hopefully, whoever is in charge of updating didn't wait until CISA gave them no choice, since in this case CISA's Binding Operational Directive was issued several days after attacks had been detected in the wild. Clearly, CISA has seen the light regarding the needed speed to respond. That chart we looked at last week demonstrated that they "get it" since many of the leaves on their decision tree required this 3- days-to-patch response time.

I really do hope that the word is filtering out that everything we have known about the dynamics of vulnerabilities, exploitation, attacks and patching has been thrown up in the air. It's unclear when and how it's going to settle down, but what is clear is that nothing will be as it has been. I'm seeing many predictions of a coming onslaught of massive AI-driven cyberattacks. As I've said, to me that seems less likely because it's unclear how that makes the attackers any money, and cryptocurrency money is entirely the name of the game today. What I expect to be more likely is many more successful network penetrations followed by extortion. And the bad guys are increasingly likely to attack those enterprises that really must protect exfiltrated data. That large law firm we recently reported on made the defensible and sane decision to pay because the cost to them of not paying would simply be too great.

The problem with changing updating habits is the great weight of institutional inertia. The hope is that the AI-attack hysteria – which I doubt will materialize – may be what the IT department needs to obtain the resource they require to be able to update far more nimbly.

Australia is disturbed by a deeply compromised infrastructure provider

Last Wednesday, Mike Burgess, the current Director-General of Security and the head of ASIO the Australian Security Intelligence Organisation, published his annual Threat Assessment for 2026. It was not cyber-specific, talking about many other social aspects which impinge upon security. But there was a section regarding threats to Australia's Critical Infrastructure, and it was a doozy. Mike wrote:

> *Critical Infrastructure: The third matter we dealt with can also be a 'threat to life' in extreme circumstances. We discovered nation state hackers had compromised the network of an Australian critical infrastructure provider. ASIO assessed the hackers were preparing for sabotage. They weren't planting 'digital dynamite' as such; they were mapping out the*

network and maintaining access so they could cripple it at a time of their choosing. Cyber sabotage is an evolving threat, and I have established dedicated teams to counter it. As ASIO's understanding grows, so does our level of concern.

The scale of this activity – led by one nation state in particular – is difficult to overstate. You and they would be surprised how extensive our warrant coverage is. We struggle to find a single country in our region that has not been compromised by this state's cyber apparatus. Critical infrastructure in the energy and communications sectors, as well as infrastructure supporting the military, are top targets.

In this case, a state-sponsored group didn't just achieve access to the Australian critical infrastructure provider, it successfully acquired credentials – log in details and passwords – for active users of the networks, including the IT professionals guarding it. ASIO identified, tracked and attributed the hack, and worked with the victim company and our security partners to remediate the compromise – work which is ongoing.

I encountered this report after fully digesting and laying out this week's main topic. So when I saw the way the intrusion into Australia's infrastructure provider was described, I noticed that it exactly corresponded to what we'll be examining as today's main topic. I would not be the least bit surprised to learn that they are one and the same.

OpenAI plans to "Patch the Planet"

Last Monday, not to be outdone by Anthropic with Mythos and Fable 5, OpenAI announced their initiative dubbed "Patch the Planet." This uses their Daybreak system and has already been producing results, which I'll share in a minute. Their announcement said:

We are introducing Patch the Planet, a Daybreak initiative built with Trail of Bits to help maintainers strengthen the critical open-source software the world relies on. We're pairing AI-assisted security research using our most cyber-capable models with expert human review to not only identify vulnerabilities, but help patch them.

AI is accelerating vulnerability discovery, but discovery alone does not protect users. Many maintainers are already being asked to sort through more reports, more quickly, with the same limited time and resources. Patch the Planet is built to reduce that burden, not add to it: security engineers review findings before they reach maintainers, work with projects to develop patches and tests, and build reusable workflows that help teams continue improving security after the first fixes land.

Trail of Bits has committed their entire security research organization towards this effort for our initial surge. They are working directly with maintainers to investigate and validate vulnerabilities, develop and test patches, and coordinate disclosure of vulnerabilities.

Additionally, we will be partnering with HackerOne and Calif who are helping us take our efforts further with vulnerability triage, coordinated disclosure, and additional focused vulnerability discovery efforts.

How Patch the Planet works: Each engagement under Patch the Planet begins in consultation with the maintainer. For each collaboration, security engineers work with maintainers to understand each project's needs, preferences, and where additional security effort would be most useful: vulnerability validation, patch development, CI/CD improvements, or longer-term security engineering. Once aligned, researchers investigate potential vulnerabilities, validate meaningful issues, develop or refine patches, support testing, and coordinate disclosure through the project's established channels.

Initial participants include cURL, NATS Server, pyca/cryptography, Sigstore, aiohttp, the Go project, freenginx, Python, and python.org. These projects support widely used networking, cryptography, software supply chain, and language infrastructure, where stronger security can benefit a broad range of downstream products and services. Additional projects will join in future rounds.

Security researchers are equipped with our frontier models as well as Codex Security to support the analysis, patch development, testing, and documentation. Participating projects receive access to ChatGPT Pro; conditional access to Codex Security; and API credits for core open-source development, maintainer automation, and release workflows. Trail of Bits has developed AI-assisted workflows for deduplication, triage, and patching that projects can run with this support.

Trail of Bits has dedicated security engineers to work full-time with Codex and GPT-5.5-Cyber across 19 open-source projects, and has already identified hundreds of security issues and merged dozens of patches, with many more still undergoing coordinated disclosure.

The initial sprint also produced reusable security infrastructure: fuzzing harnesses, historical-CVE analysis pipelines, differential-testing systems, threat models, expanded test suites, and workflows for deduplication, false-positive filtering, severity correction, and patch generation. Some project-specific details will be shared later as testing, remediation, and coordinated disclosure progress. A few early examples show what the team was able to build and find:

A fuzzing lab in less than a day. Trail of Bits engineers used repeated Codex /goal runs with GPT-5.5-Cyber to build an entire fuzzing lab covering dozens of entry points, variant builds, platforms, and novel test seeds. Engineers set the objectives and refined the prompts; the system then used coverage feedback to keep expanding into new surfaces, target edge cases, and filter weak or invalid candidates.

Trail of Bits engineers found that, with limited guidance, GPT-5.5-Cyber made useful choices about where to expand coverage, which builds and entry points to probe, and which candidates were too weak to pursue. The completed setup took less than a day. Trail of Bits estimates that building the same lab manually would ordinarily take at least several weeks.

A reusable pipeline for finding variants of known vulnerabilities. The team built an end-to-end system that ingests historical CVEs, extracts relevant vulnerability patterns, searches target codebases for related flaws, and sends candidate findings through specialized judging agents. The pipeline deduplicates results, filters likely false positives, and routes the strongest evidence to security engineers for manual confirmation.

This turns years of public vulnerability history into a repeatable search strategy that can be applied across projects. Trail of Bits found the models especially effective at this kind of variant analysis, which uncovered many additional issues across the codebases under review.

If this was posted this time last year, these details would have left our mouths hanging open in both wonder and disbelief. But now, today, our reaction is "Okay, sure, what else?" And there is more. They wrote:

> *Differential testing in days instead of weeks or months. Different implementations of the same protocol should usually behave the same way under the same inputs. When they diverge, one may contain a bug. Applying this idea at scale is normally difficult because engineers must write custom shim and glue code connecting each implementation to a common test harness.*
>
> *Codex generated and iterated on that code, allowing multiple implementations to be fuzzed against one another and their behavioral differences investigated.*

Notice that we are hearing the terms "repeated" and "iterated" more and more. What we're collectively learning is that AI gets better when it iterates on problems. They continue:

> *The workflow filtered many weak or invalid results and produced a comparatively high-signal set of candidates for expert review. The team reached those results within days, compressing work that has historically taken weeks or months. Trail of Bits is continuing to expand and refine these tests before publishing project-specific details.*
>
> *Testing software against the behavior its specifications promise. The teams used Codex to develop threat models, attack taxonomies, invariant tests, and property-based tests grounded in project specifications and RFCs. These methods exposed notable differences between intended and actual behavior while leaving projects with broader test coverage, stronger documentation, and improvements to CI/CD and software-supply-chain tooling.*
>
> *Security engineers reviewed every finding before it reached a maintainer. Trail of Bits engineers manually reviewed every security issue before it was submitted to a maintainer, and the added value of this step cannot be understated. While frontier AI models are highly capable of finding vulnerabilities and patching them, they also produce a high volume of false positives that can contribute to the already overwhelming backlog maintainers are facing.* ***Patch the Planet*** *solves for this by having dedicated Trail of Bits researchers reproduce the evidence, check findings against project-specific documentation and threat models, remove duplicates, reassess severity, and prioritize confirmed vulnerabilities for remediation. They also develop and submit patches in accordance with maintainers preferences. Maintainers remain in control of what patches are deployed and how disclosure is handled.*
>
> *What OpenAI Daybreak is already finding: Patch the Planet builds on a broader body of Daybreak work showing how frontier models can help defenders find, validate, and remediate serious vulnerabilities in widely used software.*
>
> *We are sharing a few early highlights here, while withholding exploit mechanics and project-specific details where disclosure is still underway. As fixes land and coordinated disclosures conclude, we plan to publish deeper technical reports that walk through individual findings, research methods, validation workflows, and lessons other defenders can apply.*
>
> *Our findings span every layer of the software stack, with many more still in the disclosure process.*

Operating systems

- *Linux Kernel: GPT-5.5-Cyber identified security-relevant components across more than 30 million lines of code, flagged potential security issues, and then validated them dynamically, generating 8 kernel pointer information leak proof-of-concepts (PoCs) and 24 local privilege escalation exploits. We note that hundreds of issues were identified, this is the subset for which PoCs were automatically generated.*

- *OpenBSD: Our models identified a 23-year-old use-after-free in OpenBSD's kernel implementation of System V semaphores. OpenAI researchers reproduced the issue and confirmed that it could allow an unprivileged local user to escalate privileges to root.*

- *FreeBSD: Security researchers at Calif used Codex to find and validate using proof-of-concept exploits for several LPEs in FreeBSD. Across a broader FreeBSD campaign, OpenAI researchers confirmed 34 vulnerabilities and produced 7 local privilege escalation PoCs.*

Networking

- *dnsmasq: Codex Security independently identified vulnerable patterns corresponding to four of the six dnsmasq CVEs later fixed in 2.92rel2.*

- *HTTP/2 Bomb: Calif used Codex to identify "HTTP/2 Bomb," a denial-of-service technique affecting major HTTP/2 implementations including NGINX, Apache, IIS and Pingora. Calif's analysis suggested that more than 880,000 Internet-facing websites were running affected server software with HTTP/2 enabled.*

Well, ***that*** is interesting and deeply annoying. Those are the jerks we looked at a couple of weeks ago, who bragged about the discovery of this protocol failure vulnerability and released its information, including a proof-of-concept, in a complete lack of coordinated disclosure. They essentially said "AI has changed everything such that coordinated disclosure timelines no longer apply." Meanwhile, those in charge of web server operation were scrambling in a panic which could have been avoided with just a little bit of courtesy. I'd love to see Calif's access to Daybreak rescinded since that's not the way it was supposed to be used. OpenAI continues:

Browsers

- *Chrome: OpenAI researchers found and reported 5* ***exploitable*** *vulnerabilities in Chrome's V8 JavaScript engine, including three that were identified and remediated within days of being introduced.*

- *Safari: In roughly a week of focused WebKit work, over 10 exploitable Safari vulnerabilities were found and reported.*

- *Firefox: OpenAI Preparedness identified a WebAssembly vulnerability (CVE-2026-8390) with GPT-5.5 during safety evaluations that Mozilla patched two days before Pwn2Own Berlin, prompting 5 of the 6 registered Firefox entries to withdraw. No Firefox exploit was successfully demonstrated at the competition.*

Wow. Isn't THAT cool! This is what tightening up the world's software is going to look like.

Open-source software is shared infrastructure. Securing it should be shared work. AI is changing the pace of vulnerability discovery, and the work now is to make sure the benefits reach the maintainers and users who need them most.

Patch the Planet is designed to put that full defensive loop in service of maintainers: discovery, validation, severity review, disclosure, patch development, testing, and deployment. Frontier models can make parts of that loop faster, but the aim is to give the people responsible for shared infrastructure better tools and more capacity, while preserving their agency over how changes land.

This first sprint shows what sustained collaboration among maintainers, security engineers, and AI-assisted workflows can produce: immediate fixes, stronger project infrastructure, and reusable security work that can continue improving open-source software over time.

This is just the beginning. As more fixes land and coordinated disclosures complete, we plan to publish deeper technical reports on selected findings, the methods used to discover and validate them, and the workflows defenders can adapt to help protect the software everyone depends on. If you are a maintainer, you can apply to join Patch the Planet.

And I have the link to apply to "Patch the Planet" here in the show notes: https://trailofbits.com/patch-the-planet

So, Daybreak was a bit delayed relative to Claude Mythos Preview. And it appears that, as we might expect, its approach differs in the details. But the evidence clearly suggests that OpenAI is not out of the game by any means. And that's great news for everyone.

Meta's really bad idea bites them in the butt

We briefly talked about Meta's clearly-misguided plan to record all of their employees' keyboard, mouse and screens for the ostensible purpose of training AI of some sort. At the time I quipped that it would be weird to have AI looking over our shoulders training on our work — seemed like training our own replacement. But in classic "what could possibly go wrong" failure, it was worse than that. Last Monday, WIRED picked up and covered the adventure under their headline: *"Meta Exposed Data Internally From Its Controversial Employee-Tracking Program"* and the teaser: *"Employees had previously raised concerns about the initiative, which involves collecting workers' keystroke data to train AI models."* WIRED wrote:

Meta left potentially sensitive information collected from employee laptops accessible to anyone inside the company, according to an internal security notice seen by WIRED and three current employees familiar with the issue. The data, which was collected as part of a divisive initiative to train artificial intelligence models, is believed to include keystrokes, mouseclicks, and content displayed on the computer screens of Meta's US employees.

Meta spokesperson Tracy Clayton initially confirmed to WIRED that the company is investigating the security issue. As this story was being published, he added that Meta is pausing the data collection program indefinitely. Clayton said: "We have carefully designed this program with privacy safeguards and while we have no indication at this time that any data was improperly accessed by Meta employees, we're pausing it while we investigate."

According to documents viewed by WIRED the security notice sent out Monday indicated that "employee data across 45,000 hive tables," had been exposed. Those tables included employee activity such as "full prompts and transcriptions, private conversations, people and performance data."

Wow. Big Brother much? Basically, employees are being fully and continuously surveilled with all of that mass of data collected for "AI Research". The article continues:

Some employees at Meta quickly seized on the security failure, saying in internal forums that it validated concerns they had raised when the company began tracking workers' corporate laptops in April as part of a program known as the ***Model Capability Initiative.***

Comments about the incident posted on internal forums Monday included questions about how Meta's privacy reviews failed to prevent the breach, and whether everyone whose data was potentially exposed will be allowed to attend a meeting going over what went wrong, according to posts seen by WIRED. In one internal forum where staffers are known to trade jokes, an employee posted a meme from The Office of the character Jim Halpert holding a sign that reads, "0 days since our last nonsense." Sources at Meta, who were not authorized to speak publicly, tell WIRED the incident has now been marked as closed, meaning it was likely resolved.

In an internal post responding to employees' questions on Monday seen by WIRED, Andrew Bosworth, Meta's chief technology officer, said that the tracking program's implementation had fallen short of the standards outlined in its privacy review and that findings from the incident would be shared. Bosworth wrote: "Here we had misconfigured ACLs [access control lists] and we need to understand how that happened, track down every data access and understand it."

A couple of months ago, Bosworth told employees concerned about potential data leaks that the tracking program is "tightly controlled" and uses the same protection standards, storage systems, and access controls as other sensitive datasets, according to internal posts seen by WIRED. Last month, more than 1,600 Meta employees signed an internal petition protesting the laptop surveillance effort, warning that "collecting this data introduces both security and regulatory risks for Meta, including the potential for breaches and unauthorized disclosure." The petitioners also expressed concerns with what they viewed as a lack of safeguards that Meta had put in place. One engineer also wrote a widely shared internal note saying having their laptop screen scraped for training data without their consent felt like an invasion of privacy and amounted to exploitation.

Meta executives have previously defended the data-gathering project, saying it was necessary to train AI systems to use computer software the way humans do. In audio of a company meeting leaked last month Mark Zuckerberg told employees that "AI models learn from watching really smart people do things," and the "average intelligence of the people who are at this company is significantly higher" than the average contractor who could be hired specifically to produce this kind of data.

But after widespread protest from employees, Meta this month began offering more exemptions to the monitoring, including letting staffers briefly turn off the surveillance so they could complete sensitive tasks, such as scheduling a personal appointment, according to two people familiar with the matter. Some employees are still demanding that the tracking be stopped altogether.

Meta faces more regulatory scrutiny about data security than most companies. It is subject to a US Federal Trade Commission consent decree that expires in 2040 requiring it to maintain processes to avoid breaches. But current and former employees have told WIRED that the requirements are inadequate and outdated. Meta also has also begun offloading some work reviewing programs and features for potential privacy and security risks to artificial intelligence. It wasn't immediately clear whether AI played a role in the access control issue with the MCI data.

The security incident will likely contribute to the ongoing morale crisis at Meta, where employees have been frustrated by the past few years of mass layoffs, a turbulent reorganization, and an all-out push to develop AI models and features. In March, Meta created a new Applied AI team and moved some 6,500 employees into new roles focused on improving AI models. Some Meta staffers have described the projects they have been assigned as menial and "soul-crushing."

Bosworth sent out a memo to employees last week apologizing for the company's "atrocious" communication about the AI reorganization and promising improvements, including clearer communication and the return of some office perks.

Meta does not seem like an employee-friendly place to work, but I'll confess to being able to see both sides of this. Certainly, the idea of essentially sucking in everything every employee does is inherently creepy. And the question of its secure storage is the first thing that springs to mind. In that sense it's identical to the reception Microsoft received when they introduced "Recall." Everyone's immediate reaction was: "Uhhhhhh... and how exactly are you doing to absolutely positively keep all of our screen history safe forever?" And on top of that Microsoft had to arrange to NOT capture anything that might be sensitive, like on-screen passwords and credit card numbers.

So, before I examine the other side of this argument, just so we're very clear, I fully get it that streaming into storage somewhere, every keypress, every mouse twitch and every screen image experienced and created by a mass of employees is just asking for trouble, not to mention being an astounding invasion of privacy. In the past, we've examined the amount of (or lack of) privacy an employee using company bandwidth on company computers in a company's facility should reasonably expect, and we've seen the need for an enterprise to make whatever it's doing with regard to monitoring its own network and thus indirectly its own employees, very clear. But Meta's recorded surveillance of every twitch is taking that to extremes. One question I had was whether Mark Zuckerberg's and the other C-suite executives were also participating in this grand surveillance experiment? Or had they perhaps politely excused themselves from the same super-secure surveillance that everyone else was subjected to? After all, if the AI is supposed to be training on the smartest people available, who better than the C-suite execs at the top of the pecking order?

Okay. With the horrendous privacy consequences acknowledged, I want to explore the flip side. With a brand new technology such as these massive large language model neural networks, you really don't know what you can do until you try. Since the truth is, we stumbled upon the AI effect as much as we deliberately designed it, the past several years of explosive AI growth has been a testament to the "let's try this and see what it does" approach. We're truly feeling our way forward. Someone said "Hey, you know, when I tell the AI it was wrong it readily agrees. So

how about if, instead, we just feed its first answer back in, as a loop, and let it come up with a more refined answer the second time?" – and so was born the recent notion of iterating toward a conclusion. Sure, it burns tokens, but someday tokens will be cheap, and even now the much superior results are worth the cost.

So my point is that, aside from the worrisome privacy costs, I can see the somewhat robotic and empathy-challenged Mark Zuckerberg deciding that they should just feed everything everyone does into a massive AI and see what comes out. Could they train an AI to be a functioning Meta employee replacement? Or who knows what? But that's the point I want to convey: at this still very early stage of AI understanding and development there's just no telling what might happen. We got surprisingly capable ChatBot LLM AI by pouring the entire Internet into a model until it was able to predict that data. So what happens if we pour every click, twitch, keystroke and screen image seen by Meta's employees into another big empty model canister until it's able to predict what an homogenized Meta employee would do? What might we get? There's no telling until someone tries it. It might be an AI that mostly wants to hang out at the water cooler, or it might be able to perform useful work autonomously — and wouldn't that be something?

What does seem clear is that someone is going to do that. It's just hanging out there waiting to be done. What happens when an AI model is trained on all of an employee's inputs and outputs? Perhaps Meta is not the right place for this experiment. But I can readily defend the idea, aside from the privacy downsides, what is a mid-level employee to a corporation other than the actions they take given the inputs they receive? And can that be modeled?

And the kiddies get into the act

Our frequent show contributor, Simon Zerafa sent me a link as I was wrapping this up. His email subject was "Assorted Zero Days Dropped on Github." Simon wrote "Someone is disclosing zero days on GitHub, for assorted applications: https://github.com/bikini/exploitarium Seems like responsible disclosure is going out of fashion 😟.

I count 23 various proofs of concept across a wide range of random targets. Nothing looks earth shattering, but none are what their code's authors intended. The author of this collection wrote the following, which I thought was worth sharing here:

> *This repo was incomplete when published. That's why some findings are kinda ass (ghidra) and some are better. Going forward, only serious vulnerabilities will be shared (Floci, libssh2, FFmpeg, c-ares).*
>
> *In regard to AI usage, my fuzzing workflow was automated by AI with a strict harness. I used GPT-5.5-3-Codex-Spark for ALL the fuzzing, as barely any "thought" is necessary when provided with an efficient harness. Contrary to the growing narrative that I'm just some random child burning tokens, I DO actually have a degree in the subject and have published multiple papers on fuzzing methodology. I spent years researching and developing new tools and ideas for how to fuzz. You do NOT need a SOTA model to help you identify these issues, I promise! While being able to afford a better model is helpful, my data seems to show that it is only marginal when paired with decent human oversight and a good harness. None of the actual PoCs themselves were vibe-coded; I did, in fact, hand-type them.*

I did use AI assistance for writing the PoC for RustDesk, however, as I'm not as familiar with the language. The README files are very clearly entirely AI, however, as AI can format a pretty mean Markdown file. I reviewed them to make sure they were accurate.

I'd also like to credit someone for the objdump finding. It turns out, someone beat me to the punch (they also have a better PoC too!). Please give them the credit they deserve: https://github.com/4D4J/objdump-Out-Of-Bounds-write

What this demonstrates so clearly is that we have entered a new world where the bar has been lowered so far that vulnerabilities are no longer either difficult or expensive to discover, and this dramatically reduces their perceived value. This means that an entirely new cohort of what we might have once referred to as "script kiddies" are now able to "script" AI to play in what was previously an "experts only" sandbox. And since these new participants may lack the training, discipline and reverence that accompanies hard work, they are, as Simon noted, tossing the previous respectful model of responsible disclosure out the window. They don't value their own discoveries because they came by them too easily. They're much more interested in showing off.

Aside from the consequences of the cost of vulnerability discovery being reduced to near zero, what this individual has to say about their ability to use lower-ranked models to obtain useful results is certainly interesting, too. And it fits with our general sense that AI was able to obtain such results earlier than we knew... we just hadn't yet figured out how to ask it the right way.

After our collective attention woke up to the realization that AI could do that, with the concomitant "Oh crap! What if the bad guys jump on this too?" — everyone switched into high gear and the race has been on to further figure out and fine-tune AI vulnerability discovery and to then shore up our historically flaky software before it can be exploited.

What an amazing time to be here! :)

The AI Corner

For this week's AI Potpourri, I want to share what Andrew Ng recently wrote in his DeepLearning newsletter regarding the focus that's currently gripping the AI community. It serves to further reveal the nature of current AI and everything about it also feels deeply intuitively correct to me. Here's what Andrew wrote:

> *Dear friends, "Loop engineering" is a hot buzzphrase after mentions of it by Boris Cherny (Claude Code's creator) and Peter Steinberger (OpenClaw's creator) went viral on social media. Loops are now a key part of how we get AI agents to iterate at length to build software. In this letter, I'd like to share my 3 key loops for building products. These loops guide not just how I build software, but also how I decide what software to build.*

I'll interrupt briefly to explain that Andrew's three loops represent the three typical and distinct phases of any product creation process. Someone specifies what the goals are. Then those goals are coded. Then the original specifier, seeing the initial actual results of their specification may change the spec and re-code. And then once the product is placed into use, feedback from the field may be used to further refine the result. This wasn't clear to me initially, but it should help to understand what Andrew means by "loops". He continues:

> ***Agentic coding loop:*** *Given a product specification and optionally a set of evals (that is, a dataset against which to measure performance), we can have an AI agent write code, test its work, and keep iterating until the code is bug-free and meets its specification. This idea of closing the loop took off around the end of last year, and it has been a game changer in enabling coding agents to work longer productively without human intervention. For example, over the weekend, I was building an app for my daughter to practice typing, and my coding agent could easily work for around an hour, using a web browser to check what it had built multiple times before getting back to me, without needing my intervention.*
>
> *The engineering loop executes quickly. Every few minutes, the coding agent might build and test a new version of the software. I hear frequently from developers who are finding new ways to engineer more effective engineering loops. This is an active area of invention!*
>
> ***Developer feedback loop:*** *In this loop, a developer examines the current product and steers the coding agent to improve it. Last year, a lot of developers (including me) were acting as the QA (quality assurance) function for our coding agents, manually finding bugs and then asking the agent to fix them. But with coding agents much more able to test their own code, the amount of time we need to spend on this function has decreased significantly. This allows us to make higher-level product decisions, such as what key features to offer, where the UI needs improvement, and so on.*
>
> *The developer-feedback loop operates over time intervals between tens of minutes and hours — that's how frequently a developer might review a product and give feedback. In the case of the typing app, I changed my mind a few times about the visual design, what cat costumes she can unlock as she learns (she loves cats), and the user flow for a grown-up to log in and steer the child's learning experience.*
>
> *When a developer has a clear vision for what to build, it is still a lot of work to translate that vision into a specification for a coding agent to implement. Further, after the developer has*

seen an implementation, they might update (or perhaps clarify) the spec to steer it toward what they want. If you find that the system repeatedly runs into certain problems, building a set of evals for the agent becomes useful.

AI-native teams are increasingly using AI to help shape product direction, for example, automating the gathering and analysis of usage data, summarizing written and verbal customer feedback, or carrying out competitive analysis. However, for pretty much all the products I'm involved in, I see humans as having a significant context advantage over current AI systems — we know a lot more than the AI system about the users and the context the product has to operate in — and thus humans play a critical role. Many people describe this human contribution as "taste," but I prefer to think of it as humans having a context advantage, since that gives us a clearer path to helping AI systems get better. This also speaks to why this step can't be automated: So long as the human knows something the AI does not, human-in-the-loop is needed to to inject that knowledge into the system.

***External feedback loop:** This includes a wide range of tactics like asking a few friends for feedback, launching to alpha testers, or putting the code into production with A/B testing. These tactics are usually slow, rarely taking less than hours and sometimes taking days or even weeks. This data informs the developer vision, which in turn continues to drive the detailed product spec, which in turn drives the coding agent.*

With coding agents speeding up software development, more engineers are starting to play a partial product management role. For many engineers who are growing into this role, the hardest part is shaping the product vision and striking a balance between building (bridging the gap between vision and spec) and getting user feedback to evolve the vision. It is important to do both!

I will write more about how to do this in future letters, but for now, I find it encouraging that engineers are playing an expanded role (just as product managers and designers now do more engineering). Keep building! /Andrew

One of the oddities we've often seen from today's AI is that it can be wrong. But then, when it's shown its mistake it will easily see that it was wrong. We're all used to computers being completely deterministic. A pocket calculator – a simple form of computer – doesn't give us different answers each time we input the same series of calculations. But today's AI does. This has been both disconcerting and puzzling to those of us who have been using conversational AI for a while. It's similarly confusing that after AI produces some code we can feed that same code back into that same AI and it may very likely discover some bugs in the code it just wrote.

So the dialog would go: *But, wait a minute, didn't you just write that code and you were presumably completely happy with it when you gave it to me? But now, when I give it right back to you, you're saying "oh look, I found some bugs!" but, you just produced that code!*

This is another way for us to understand why Mozilla's early use of the Claude Mythos Preview may have missed a few bugs in Firefox while discovering hundreds more. It would have probably been worthwhile to ask Mythos for exactly the same thing a few more times. No traditional computer nor any calculator would ever behave in this fashion. But then again, neither are we able to have what passes for a conversation with any traditional computer or calculator.

We know that in order to make neural nets work, it's necessary to "jumble them up a bit" by deliberately injecting some noise into the system. In searching for a clear physical analogy to visualize this I was reminded of trying to fill a bottle with too many pills. If you fill the bottle to the top, no more pills will fit in. But if you then tap the bottle on the counter or shake it sideways a bit – sure enough, the pills that are already in the bottle will further "settle" to open additional space at the top for more. Mathematically, we would think of I as "finding a minimum" which might require rearranging some previously arranged pills for better packing. In much the same way, a neural network can "find a better minimum" when it's shaken up a bit through the injection of some noise.

But the necessary consequence of this noise injection is that the final output of a massively complex neural network will be different each time it's used, even when giving identical inputs. The same number of pills in the bottle, but a different packing arrangement each time you fill it.

So this discovery and practice of "looping" is a significant win and improvement. It explicitly recognizes that "asking again" is an important and meaningful step in the evolution of our understanding of how to obtain the most value from these crazy new non-deterministic AI neural nets. Of course, under the "there's no such thing as a free lunch" rule, each round of looping burns up additional tokens, so the cafeteria bill can wind up being high.

Being a strong proponent of local AI – despite it not being super-practical this instant – I'll note that we do not typically require finished code in only minutes or hours. Andrew's typing practice app for his daughter will likely see many months of use. So waiting a few days for a very much slower local AI to "loop-out" a mostly finished product incurs very little cost while producing something of quite enduring value.

Miscellany

Remembering Kevin

I'm going to share a not widely known story about a legendary hacker friend of ours and a very good guy. The story was published last Monday in, of all places, "The Drive" dot com. Since sharing the story's headline would give away its heart warming point, I'm going to skip that. The story goes:

> *If you're any kind of car geek, you have a wild gift car fantasy. You meet a bitter divorcee who gives away an ex's prized machine out of pure spite; or maybe the guy whose tire you stopped to change turns out to be a flip-flop billionaire who rewards you with your exact spec because it's simply collecting dust that week, and hey, you stopped; your humanity's worth a Dodge Viper to a guy who can afford to run a bidet on day-old moon water, or something. OK, that one might be mine.*
>
> *But for this, it'll help if you know the name Kevin Mitnick. He was a hacker-turned-security consultant who, later in life, helped shape the modern white-hat. Just how prototypical was Mitnick? He put himself on the proverbial map in 1979 by dialing into a software company's server and copying its forthcoming operating system release in its entirety. Imagine convincing a Microsoft server to cough over an early copy of Windows 12 using little more than a phone number.*

Some online criticism implies that Mitnick was more of a social engineer than a "hacker" in the sense that we distinguish them today, but the reality is that a great deal of "hacking" is still dependent on an authorized user making a mistake—usually by revealing sensitive login data. For a reasonably realistic take on modern black-hatting, I recommend Mr. Robot; be warned, that series is heavy.

So, how do we get from old-school hacker to wild gift-car fantasy? In this case, by way of 14 counts of felony wire fraud. That's where Shawn Nunley comes in. Back in the '90s, Nunley worked for Novell, a now-defunct brand that produced enterprise software—server operating systems, messaging systems, that sort of thing. GroupWise is probably its best-known brand among the general public today, but the juicy target back then was NetWare, which was the backbone of many a corporate/government/academic network. Naturally, this made it a valuable target for a hacker like Mitnick.

Nunley wrote: "Back in the 90s, Kevin was trying very hard to hack into Novell's network. I was a network administrator. Of course, we had no idea it was Kevin, but things were happening that made it fairly obvious we had a persistent threat. Phones ringing sequentially throughout the building (war dialing) and all sorts of other signs… we knew something was up."

This was Mitnick, using a slightly more sophisticated version of the same tactic that earned him his first big score in 1979.

Nunley wrote: "Late one night at home, I got a phone call from a Novell employee named Gabe Nault. The 'employee' wanted direct inbound dial access. Since I was responsible for the entire network's inbound connectivity, I knew this type of request was abnormal and against policy."

And Mitnick, no amateur, had obviously succeeded in extracting at least some private information from Novell employees prior to his Hail Mary phone call:

Nunley said: "…this guy had a story about working on a top-secret Novell project called Snowbird (which was real) and needing to make some emergency code changes, but he was on vacation in Vail at a hotel. He needed the coveted, policy-breaking, direct inbound modem access. Right. He even mentioned his vacation in Vail, which conveniently matched the greeting on Gabe Nault's voicemail. But it all felt wrong. With a feeling of suspicion creeping in, I played it cool. I said, 'Hey man, I'd love to help you out, but I can't do what you want from here at home anyway, so I'll have to do it in the morning as soon as I get to the office. But in case I forget, please leave me a voicemail.' He agreed, and that was that."

"When I got to work, the voicemail was there, and I immediately recorded it onto a cassette recorder for safekeeping. That recording became the primary evidence in Kevin's case."

When Mitnick was caught, that's when Nunley learned that the voicemail was the only meaningful evidence that the Justice Department had against him. At first, he was on board with the prosecution, but after 5 years of repeated trial delays, Nunley grew weary of the way the law was treating his adversary, and he refused to continue working with the DOJ. Shortly thereafter, Mitnick took a plea deal and was released.

When he got out, Mitnick contacted Nunley to apologize. Their bury-the-hatchet moment was even immortalized by Wired Magazine, and they went on to become good friends.

Mitnick was barred from selling the story of his legal entanglements for seven years after his release, invoking legal precedent intended to curb profiteering by serial killers. But Mitnick was able to find plenty of work teaching people how to defend against the intrusion tactics he'd spent decades refining. He would go on to found two consulting businesses, one of which his family still owns and operates.

And now we get to the point of this story, which Nunley posted last week on Reddit:

When Mitnick passed away from pancreatic cancer in 2023, he left Nunley a gift—enough to buy his dream car, a 911 Carrera 4 GTS.

Nunley wrote of his friend: "I have had a wonderful time watching him develop into a real man. I am truly sad he is gone as he was a big part of my life for the last quarter century."

That's certainly the Kevin that we and the rest of the world came to know.

A SOTA State-Sponsored Campaign

We initially covered the so-called "FortiBleed" attack last week. At that time, the first thing I wanted to clarify was that the thing that was "bleeding" was not directly any FortiNet device but the discovery of an online unprotected database of previously bled or brute forced or hash-cracked authentication usernames and passwords. And there were around 74,000 of them reported. So that was bad.

It turns out, it's worse than we knew. And when we take in the full scope of what was discovered, what's revealed is a massive state-of-the-art automated state-sponsored scale campaign with a scope that would be difficult to overstate. This elevated the campaign to the level of this week's primary topic since everyone should understand what's going on "out there" on the big wild Internet. And it's significant to appreciate that we only know about any of this due to a configuration error on the part of a database's access controls. What else of a similar nature is almost assuredly happening that we are not aware of?

So, let me start with a report in the cyber security press, which read:

FortiBleed, a massive hacking campaign that targeted Fortinet devices this year, was far more sophisticated than security researchers initially thought. Initial reports painted the picture of a campaign that gained access to Fortinet devices, collected credentials and authentication hashes, cracked the hashes, and then the data mysteriously leaked online.

The reality is that the campaign was far more complex and targeted many more things than just Fortinet devices. Compiling data from reports published by Fortinet itself, SOC Radar, CloudSEK, Palo Alto Networks, and Prodaft we gain a much clearer picture of a broad hacking campaign that began in February this year as an internet mass-scan & brute-force operation.

Initial attacks targeted technologies such as RDWeb, Sophos and Citrix SSL VPNs, exposed RDP instances, and MSSQL databases.

The operation eventually transitioned into targeting Fortinet FortiGate firewalls, every e-crime group's favorite device, and the brute-force scans also evolved into

Protocol / Service	Port
TACACS+	**49**
Kerberos	**88**
RPC / NetBIOS / SMB	**135, 139, 445**
LDAP / LDAPS / Global Catalog	**389, 636, 3268**
SMTP / POP3 / IMAP	**25, 110, 143**
FTP / Telnet	**21, 23**
RDP	**3389**
WinRM (HTTP/HTTPS)	**5985, 5986**
MSSQL / MySQL / PostgreSQL	**1433, 3306, 5432**
RADIUS (Auth/Acct)	**1812, 1813, 1645, 1646**
FortiClient / Custom	**6162**

actual exploits that abused old and unpatched vulnerabilities (CVEs mentioned in the Fortinet report) to bypass authentication and gain control over the devices.

The attacker collected plaintext passwords from Fortinet configs, but sometime in May they also started deploying a novel script that intercepted traffic going "through" the firewalls. The script, which researchers named FortigateSniffer, targeted 24 internet protocols. The threat actor extracted anything that looked like credentials, tokens, secrets, and authentication hashes on those protocols' ports (see the table above).

The attacker also took these password and other authentication hashes and fed them into a GPU-based cluster to crack them back to their plaintext versions. The passwords were then validated inside hacked companies' networks, first to confirm them, then later to expand the attacker's access. Then, the network access was sold to other groups.

While the initial FortiBleed coverage focused on the 74,000 leaked Fortinet device passwords that were found online inside an open directory on a web server, there were EVEN MORE passwords collected through this operation by the attacker that we don't know about.

All of this was done with a custom-built attack server infrastructure that impressed most of the people writing reports about it. The entire operation is believed to be the work of a Russian-speaking threat actor who specializes in breaching networks and then selling access to them to other groups. Security firms call threat actors like these "initial access brokers."

Although several security firms have also reached the same conclusion, it was only PAN's Unit42 who named the attacker as an individual going online as SantaAd. According to SOC Radar, the "threat actor behind the FortiBleed campaign remains active" and "portions of the infrastructure continue to operate at the time of writing."

Okay. So that gives us a good overall sense for what's been going on. Palo Alto Networks added some additional information under their headline: "Threat Brief: Mitigating Large-Scale Credential Attacks" – which is certainly what this turned out to be. They wrote:

Unit 42 is aware of a large-scale password spraying & credential theft campaign ("FortiBleed") against Fortinet devices. We observed attempts targeting MSSQL devices as well, and have seen reports of Sophos devices also being targeted. While this activity is not targeting Palo Alto Networks devices, Unit 42 has observed suspicious login attempts in customer telemetry and we are providing this report out of an abundance of caution to ensure our customers have the latest intelligence and recommendations to protect, detect and respond to attacks to their network.

The threat actors are using a curated password list to attempt password spraying against services exposed to the internet. Unit 42 assesses that the initial password list for this activity was likely developed through a mix of previous breaches, including the successful exploitation of vulnerabilities. Once they obtain credentials, they add them to their password list for future attempts against additional targets, as well as for logging into accounts they successfully compromised.

The threat actors are leveraging a multi-stage process to gain persistent, high-privilege access:

- *Password spraying for initial access: Massive internet-wide scanning and password spraying attempts against Fortinet, Sophos and MSSQL services*

- *Configuration Extraction: Depending on the permissions of their initial access, the actor may exploit a privilege escalation vulnerability prior to pulling device configuration files, including stored credentials*

- *Offline Cracking: Offline password cracking of the stolen credentials adds to the password list used in step one to target new devices, as well as to log into compromised devices to establish persistence as an administrator*

Unit 42 observed an initial access broker (IAB) on the Russian-language cybercrime forum Exploit[.]in claiming responsibility for this campaign, referencing a CVE, and offering the harvested credentials for sale on June 16, 2026. Unit 42 has not validated their claims at this time. Unit 42 recommends auditing remote access logs for suspicious activity with a focus on successful logins shortly after large volume password failure events. We also recommend reviewing and implementing the hardening guidance below for edge devices. ***SOC Radar*** *provided the initial reporting on the targeting of FortiGate devices. We observed attempts targeting MSSQL devices as well, and have seen reports of Sophos devices also being targeted.*

Okay. And that leads us to the SOC Radar people who gave this the "FortiBleed" name. The headline for their reporting was "FortiBleed: SOCRadar's Investigation into 86,644 Compromised Fortinet Firewalls." They wrote:

Fortinet FortiGate firewalls and VPN gateways are among the most widely deployed network security devices in the world, relied on across every sector to control access and protect infrastructure. SOCRadar researchers found a threat actor systematically compromising them at scale, building a verified database of working credentials across 194 countries. Security researcher Volodymyr "Bob" Diachenko first flagged the exposed attacker server, and SOCRadar independently discovered and analyzed the full operation.

We were among the first to dig in, and the first to call it FortiBleed. The name stuck. This is an active breach. It has been running since at least February 2026, with more than 80,000 targets identified and thousands of devices still being actively sniffed. Its discovery started the way these things do: an exposed server, an open directory someone forgot to lock. That thread led us to 260 operational servers tied to the campaign, wider visibility than anything reported elsewhere.

The SOCRadar Threat Research Unit (STRU) spent five days on the actual data, not just the headline numbers: which sectors, which regions, how credentials were collected and cracked, and why a firmware update alone didn't close the door for most victims. While STRU mapped it, the rest of the team notified every affected customer we could reach, stood up a free checker, and pushed the full dataset to CERT and CSIRT teams worldwide. Most of it was manual, and we're still getting back to everyone who asks for their data.

This is still an active, developing campaign. Today we're publishing the full thing, as we've mapped it so far.

In the course of monitoring active threat actor infrastructure, SOCRadar threat researchers detected the operational server behind the FortiBleed campaign — a hacking group that had been quietly breaking into corporate Fortinet FortiGate firewalls and SSL VPN gateways on a massive, global scale.

The attacker's database contains login credentials for more than 86,644 FortiGate firewall devices belonging to companies and government organizations across 194 countries. These are not random guesses. These are verified, working usernames and passwords, tested and confirmed by the attackers themselves using automated tools running around the clock.

If your organization uses a Fortinet FortiGate firewall or SSL VPN product and appears in this dataset, treat your network perimeter as already compromised and act immediately.

The FortiBleed operation is built around full automation. The operation runs in two self-reinforcing stages. Stage one is credential reuse: attackers assembled usernames and passwords from earlier Fortinet-related breach dumps and infostealer malware logs, then tested them automatically against internet-facing FortiGate devices around the clock.

I'll interrupt here to remind everyone that while it's always easy to armchair quarterback after the fact, we have noted for many years that both credential spraying and brute force attacks are so easily detected. If any IT person worth their salt were monitoring their VPN firewall's authentication system and observed attempt after attempt failing, and assuming that logging in required just a username and password, the proper course of action, depending upon the value of the network that lies behind the firewall, might well be to disconnect the public-side network connection. The risk of some 24/7/365 credential-guessing attacker getting lucky might be just too high.

The question this inspires is: Does FortiGate's VPN system offer that feature? If it does not, then shame on them. If it does offer a brute-force detecting VPN lockout feature that was not enabled then shame on the IT staff who configured the VPN gateway.

But one way or another, we're seeing 86,644 usernames and passwords that were actually and truly obtained through just trying and trying. Since we know that they were also verified, we know that no other 2nd factor of authentication was required. And we also know that no control or awareness over massive numbers of failed login attempts was present. Not for any of those 86,644 endpoints. And that's really quite pathetic in this day and age. SOC continues, writing:

Stage two is passive harvesting: once inside a device, it is used as a listening post – SSL VPN traffic passing through is monitored and additional credentials are collected. Those credentials feed back into the scanner, compounding the breach. The system is entirely self-sustaining.

*One Fortinet vulnerability that has also drawn attention in connection with FortiBleed was CVE-2026-24858. Disclosed by Fortinet in January 2026, it is a critical FortiCloud SSO SAML authentication bypass with a CVSS score of up to **9.8**. Some researchers have discussed whether it may have contributed to initial access in a subset of cases, though this remains under investigation. FortiBleed is primarily a credential reuse campaign, not a zero-day exploitation event.*

The password list is not random. It is a carefully assembled collection of credentials leaked from Fortinet FortiGate devices in earlier incidents — meaning many targets may have never changed their passwords after a prior breach. The attackers know this, and they are counting on it.

The FortiBleed attackers made mistakes. Their server was left exposed with a trove of operational files that revealed far more about them than they intended. Among the recovered data were credentials for what appears to be a defense industry VPN endpoint, suggesting the group's ambitions extend beyond purely financial targets.

The tooling, infrastructure choices, and victim selection, heavily weighted toward organizations in NATO member countries, are consistent with Russian-speaking threat actors. Attribution is ongoing, but the operational fingerprints are clear.

The FortiBleed victim list spans every sector of the global economy. Among the 86,644 compromised access points identified, we found entries belonging to banks, telecom operators, hospitals, universities, government agencies, energy companies, and multinational corporations with revenues in the tens of billions of dollars. No industry was spared. No region was ignored.

Government entities alone account for 591 entries across 111 domains. Telecoms represent one of the most heavily targeted sectors with 5,616 entries. The geographic spread covers Asia, Europe, the Americas, the Middle East, and Africa. Enterprise organizations above $1B in revenue account for over 20% of all entries, representing significant financial and critical infrastructure exposure. The large share reflects smaller or unclassified organizations.

What a mess. The 4th question in their FAQ's Q&A is: *"4. Is FortiBleed a Fortinet vulnerability?"* To which they reply:

No. FortiBleed is not caused by a software vulnerability in Fortinet products. It exploits operational security failures – specifically, organizations that never rotated passwords after prior breaches, organizations using default or factory credentials, and organizations with management interfaces exposed directly to the public internet.

The attacker tests known leaked passwords against internet-facing devices. No code-level weakness in FortiOS or any other Fortinet product is required. A software patch alone will not resolve this.

We don't know how many FortiNet FortiGate VPN firewalls are currently deployed globally, so we have no way of knowing that percentage of them are represented by that number 86,644. But since that's a large number, it would be a good guess to assume that a very significant percentage of the total are there. I sincerely hope that FortiNet's FortiGate VPN designers are deeply embarrassed by the well-deserved attention this has brought to their doorstep. They should be. For the number to be that high, this cannot be a *"blame the user"* scenario. FortiNet needs to take ownership of the fact that they should clearly be doing a FAR better job of helping their users to be safe, even if they are forced to insist! (I believe it's referred to as "tough love.")